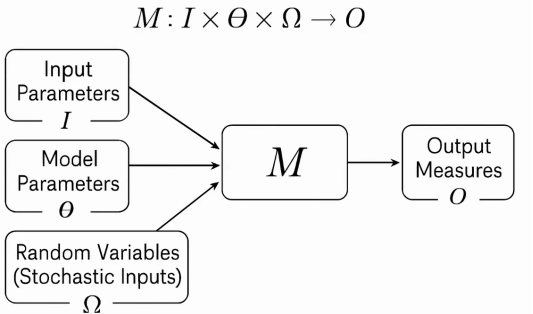


Introduction to Simulation Modelling



Definition and Core Concepts

Simulation is the process of designing a model of a real system and conducting experiments with this model for the purpose of either understanding the behaviour of the system or evaluating various strategies for the operation of the system. At its core, simulation involves creating a mathematical–logical model that captures the essential features of a system. Mathematically, a simulation model M can be represented as a function

$$M: I \times \Theta \times \Omega \rightarrow O$$

where:

- I denotes the set of input parameters
- Θ denotes the set of model parameters
- Ω denotes the sample space of random variables
- O denotes the set of output measures

```
import numpy as np
n = 1000
rolls = np.random.randint(1, 7, size=n)
prob_six = np.sum(rolls == 6) / n
print("Estimated probability of rolling a six:", prob_six)
```

Basic Concepts and Example Models

System Concepts

To make the fundamental concepts of simulation modelling concrete, let's consider our coffee shop example again: Consider a **coffee shop** with one **barista**:

- Customers arrive **randomly** with the arrival rate λ .
- If the barista is free, service begins immediately with the service rate μ .
- If busy, customers **wait in a queue** until the barista is free.
- After service, customers **depart**.

This is the simplest queue called an **M/M/1 queue**.

State Variables

The state of the system at any time t is completely described by the state vector $\mathbf{S}(t)$. **State variables** are the collection of information necessary to describe the system at any point in time.

They can be distinguished between:

- Discrete state variables:** Take on distinct, countable values (e.g., number of customers in queue, status of a server (busy/idle)).
- Continuous state variables:** Can take any value within a range (e.g., water level in a tank, temperature).

Our coffee shop The state of our coffee shop at any time t is completely described by the state vector $\mathbf{S}(t) = (L_q(t), L_s(t))$ where

- $L_q(t)$: An integer representing the **Length** of the queue at time t with $L_q(t) \geq 0$.
- $L_s(t)$: A binary integer representing the **Length** (status) of the service (i.e., the barista) at time t .
 - $L_s(t) = 1$ if the barista is busy.
 - $L_s(t) = 0$ if the barista is idle.

Event Scheduling for the Single-Teller Bank Queue

```
import heapq
current_time = 0
teller_busy = False
FEL = [] # min-heap by (time, event_type, cid)
queue = [] # list
# Schedule an event
def schedule_event(time, event_type, cid):
    # heappush keeps event_queue as min-heap ordered by time
    heapq.heappush(FEL, (time, event_type, cid))
# Event handlers
def handle_arrival(cid):
    global teller_busy
    print('Customer {} arrives at {}'.format(cid, current_time))
    if not teller_busy:
        teller_busy = True
        schedule_event(current_time, 'start_service', cid)
    else:
        queue.append(cid)
def handle_start_service(cid):
    global teller_busy
    print('Customer {} starts service at {}'.format(cid, current_time))
    schedule_event(current_time + 3, 'end_service', cid)
def handle_end_service(cid):
    global teller_busy
    print('Customer {} leaves at {}'.format(cid, current_time))
    teller_busy = False
    if queue:
        next_customer = queue.pop(0)
        teller_busy = True
        schedule_event(current_time, 'start_service', next_customer)
# Schedule customer arrivals at t=0,1,2
for i in range(3):
    schedule_event(i, 'arrival', f"C{i}")
# Event loop
while FEL:
    current_time, event_type, cid = heapq.heappop(FEL)
    if event_type == 'arrival':
        handle_arrival(cid)
    elif event_type == 'start_service':
        handle_start_service(cid)
    elif event_type == 'end_service':
        handle_end_service(cid)
def delayed_customer(env, cid, teller, delay):
    yield env.timeout(delay)
    yield env.process(customer(env, cid, teller))
env = simpy.Environment()
teller = simpy.Resource(env, capacity=1)
# Schedule customer arrivals at t=0,1,2
for i in range(3):
    env.process(delayed_customer(env, f'C{i}', teller, delay=i))
env.run()
```

Review of Probability

Random Variables

Bernoulli Random Variables: Random variable can take on only one of two possible values, often thought as success and failure

$$f(x) = \begin{cases} p, & \text{if } x = 1, \\ 1 - p, & \text{if } x = 0. \end{cases} \quad \text{where } p \in [0, 1].$$

Binomial: Random variable with parameters n and p
 $Y = X_1 + X_2 + \dots + X_n$ are independent.

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$P(k$ successes from n independent trials each with probability of p success).

$$\mathbb{E}(Y) = np, \quad \text{Var}(Y) = np(1 - p)$$

Negative Binomial: X = number of trials until k successes are obtained.

$$P(X = x) = \binom{x - 1}{k - 1} p^k (1 - p)^{x - k}, \quad \text{for } x = k, k + 1, \dots$$

Geometric: X = number of trials until a success is obtained.where k is the number of trials needed

$$P(X = k) = (1 - p)^{k - 1} p$$

Hypergeometric: X = number of trials until success, without replacement.

$$P(X = x) = \frac{\binom{k}{x} \binom{N - k}{n - x}}{\binom{N}{n}}, \quad \text{for possible } x$$

Poisson distribution:

$$P(X = x) = \frac{e^{-\mu} \mu^x}{x!}, \quad \text{for } x = 0, 1, 2, \dots$$

Uniform random variable.: A continuous random variable X is **uniformly distributed** in the interval $[a, b]$ (with $b > a$) if the probability that X belongs to any subinterval of $[a, b]$ is equal to the length of the subinterval divided by $b - a$.

$$f(x) = \frac{1}{b - a}, \quad a < x < b.$$

Normal Random Variables: Continuous distribution with mean μ and variance σ^2 .

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x - \mu)^2 / (2\sigma^2)}.$$

Exponential Random Variables: Models time between events in a Poisson process with rate λ .

$$f(x) = \lambda e^{-\lambda x}, \quad x > 0.$$

Probability Distributions Summary

Distribution	Type	PMF / PDF	Mean	Variance
Bernoulli(p)	Discrete	$P(X = 1) = p$	p	$p(1 - p)$
Binomial(n, p)	Discrete	$\binom{n}{x} p^x (1 - p)^{n - x}$	np	$np(1 - p)$
Poisson(λ)	Discrete	$\frac{e^{-\lambda} \lambda^x}{x!}$	λ	λ
Geometric(p)	Discrete	$p(1 - p)^{n - 1}$	$1/p$	$(1 - p)/p^2$
Uniform(a, b)	Continuous	$1/(b - a)$	$(a + b)/2$	$(b - a)^2/12$
Normal(μ, σ^2)	Continuous	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x - \mu)^2 / (2\sigma^2)}$	μ	σ^2
Exponential(λ)	Continuous	$\lambda e^{-\lambda x}$	$1/\lambda$	$1/\lambda^2$

Covariance and Correlation

Covariance of any two random variables X and Y denoted by

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Correlation between X and Y , denoted by $\rho(X, Y)$, is defined by

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}}.$$

```
import numpy as np
from scipy.stats import bernoulli, binom
# Simulate 10 trials
# Bernoulli random variable
p = 0.5 # Probability of success
X = bernoulli.rvs(p, size=10)
# Binomial random variable
n = 5 # Number of trials
X_binom = binom.rvs(n, p, size=10)
# Draw samples from a uniform distribution over [low, high)
- np.random.uniform(low=0.0, high=1.0, size=None)
# Draw samples from a normal (Gaussian) distribution
- np.random.normal(loc=0.0, scale=1.0, size=None)
# Draw samples from an exponential distribution with rate lam = 1/scale
- np.random.exponential(scale=1.0, size=None)
# Draw samples from a Poisson distribution (discrete events per interval)
- np.random.poisson(lam=1.0, size=None)
plt.hist(
    sample, # your data array
    bins=..., # number of bars
    facecolor=..., # fill color of the bars
    edgecolor=..., # color of the bar edges
    alpha=..., # controls the opacity (transparency) of the bars
    density=True # normalize histogram to form a probability density (area = 1)
)
# Randomly select elements from an array, useful for custom discrete events
np.random.choice(a, size=None, replace=True, p=None)
# Return random integers from low (inclusive) to high (exclusive)
np.random.randint(low, high=None, size=None, dtype=int)
# Compute the sample mean
np.mean(sample)
# Compute the sample variance (divide by n - 1)
np.var(sample, ddof=1)
# Compute the sample standard deviation (divide by n - 1)
np.std(sample, ddof=1)
np.random.seed(211)
sizes = [10, 100, 1000, 10000]
for n in sizes:
    sample = np.random.uniform(0, 1, n)
    print(f"Sample mean (n = {n}):", sample.mean())
# Compute the sample covariance matrix of X and Y
np.cov(X, Y)
# Compute the Pearson correlation coefficient matrix of X and Y
np.corrcoef(X, Y)
# The arrival time of each customer
arrival = np.array([0.0, 0.5, 3.4, 6.4, 6.6, 9.5, 9.7, 10.4, 11.4, 12.1, 12.7, 13.6, 14.2, 15.5, 18.1])
# The time service begins
begin = np.array([0.0, 0.5, 3.4, 9.2, 12.3, 15.6, 20.1, 25.3, 27.4, 28.6, 34.0, 36.0, 42.5, 43.0, 45.7])
x = ... # your x-values (e.g., arrival times)
y = ... # your y-values (e.g., waiting times)
plt.scatter(
    x, y, # x and y data points
    color=..., # dot color
    alpha=..., # dot transparency
    edgecolor=... # outline of dots)
```

Random Number Generation

Multiplicative congruential method

Definition (pseudo-random numbers). Given integers a (the multiplier), m (the modulus), and an initial seed X_0 , we define the recurrence

$$X_{n+1} \equiv aX_n \pmod{m} \quad n \geq 0 \quad .$$

The pseudo-random numbers are then $U_n = X_n/m$. **Definition (Order)** The order of an integer a modulo m , written $\text{ord}_m(a)$, is the smallest positive integer k such that **Definition (Primitive root modulo m)** An integer a is a primitive root modulo m if

$$\text{ord}_m(a) = \varphi(m)$$

where $\varphi(m)$ is Euler's totient function which counts the number of positive integers less than m that are coprime to m .

Linear Congruential Generators

$$X_{n+1} \equiv aX_n + c \pmod{m}$$

Theorem (Hull-Dobell). The LCG has full period m if and only if all three of the following hold:

Multiple-Recursive Generators

$$X_n = (a_1X_{n-1} + a_2X_{n-2} + \dots + a_kX_{n-k}) \pmod{m}$$

Testing Random Number Generators

Empirical Tests Empirical tests treat the generated sequence as data. They are essentially statistical hypothesis tests.

- Idea:** simulate a sample of n values, then check whether its distribution matches the **Uniform(0, 1)** law (in one or more dimensions).
- Output:** a test statistic and a p -value. If the p -value is small, we reject the null hypothesis.

Frequency (Chi-square) test We divide $[0, 1]$ into k equal subintervals. If n values are generated, the expected count per interval is $E = n/k$.

- H_0 : the numbers are i.i.d. **Uniform(0, 1)**, so each interval is equally likely.
- H_1 : the distribution is not uniform (some intervals more/less likely).

Test statistic:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E)^2}{E}$$

where O_i is the observed count in interval i . Compare with χ_{k-1}^2 distribution.

Serial test We form pairs (U_{2i-1}, U_{2i}) and place them in a $k \times k$ grid.

- H_0 : the pairs are uniformly distributed in $[0, 1]^2$, i.e. no correlation between successive numbers.
- H_1 : the pairs are not uniformly distributed, indicating correlation or clustering. Chi-square statistic compares observed cell counts with n/k^2 .

Gap test This test helps to determine if a sequence of random numbers exhibits sufficient independence. Fix an interval $A \subset [0, 1]$. Measure the number of values outside A between two successive "hits" inside A .

- H_0 : gaps follow a geometric distribution with success probability $p = |A|$.
- H_1 : gap lengths deviate from the geometric law.

Observed vs. expected gap frequencies are compared using chi-square.

```
# --- LCG: fill in your parameters ---
a = 33# multiplier
m = 251# modulus
c = 213# increment
seed = 3# seed
def lcg_int(n, a, m, c, seed):
    %Generate n raw integers Xn using an LCG.
    x = seed
    results = []
    for i in range(n):
        x = (a * x + c) % m
        results.append(x)
    return results
# a. Function roll_die
def roll_die(n_rolls):
    %Generate a fair die face (1-6) from a given PRNG
    # Generate sequence of n_rolls Xn
    seq = lcg_int(n_rolls, a, m, c, seed)
    results = []
    for num in seq:
        face = (num % 6) + 1
        results.append(face)
```

```
    return results
# b. Simulate 10000 rolls
rolls = roll_die(10000)
# c. Plot histogram
import matplotlib.pyplot as plt
import numpy as np
plt.hist(
    x = rolls,
    bins = np.arange(1, 8) - 0.5, # align bins with die
    facecolor = "#C486BC",
    edgecolor = "black",
    alpha = 0.6,
    rwidth = .9)
plt.xticks(range(1, 7))
plt.xlabel("Die Face")
plt.ylabel("Frequency")
plt.title("Simulated Die Rolls with the Poor LCG")
plt.show()
```

Simulating Random Variables

Inverse Transform Technique

Let X be a continuous random variable with CDF $F(x)$. Then, the random variable $U = F(X)$ is uniformly distributed on $[0, 1]$. Conversely, if U is a uniform random variable on $[0, 1]$, then the random variable $X = F^{-1}(U)$ has the CDF $F(x)$. Algorithm

- Generate a uniform random number $U \sim \text{Uniform}(0, 1)$.
- Compute $X = F^{-1}(U)$, where F^{-1} is the inverse CDF.
- Return X as the desired random variate.

Acceptance-Rejection Method

Continuous Case To generate a random variable with PDF $f(x)$, we need a proposal PDF $g(x)$ and a constant c such that $f(x) \leq c \cdot g(x)$ for all x . Algorithm

- Generate proposal** Draw Y from the distribution with PDF g .
- Generate uniform** Independently, draw a uniform random number $U \sim \text{Uniform}(0, 1)$.
- Acceptance test** If $U \leq \frac{f(Y)}{c \cdot g(Y)}$, then accept and set $X = Y$ and stop. Otherwise, reject Y and return to Step 1

the acceptance probability is $\frac{1}{c}$. Equivalently, the number of proposals until acceptance is a geometric random variable with mean c . The conditional PDF of an accepted proposal is

$$f_{\text{acc}}(x) = \frac{\mathbb{P}[\text{accept in } (x, x + dx)]}{\mathbb{P}[\text{accept}]} dx = \frac{f(x)/c}{1/c} = f(x).$$

- Discrete Case** To generate a random variable with PMF, p_j we need a proposal PMF q_j and a constant c such that $p_j/q_j \leq c$ for all j . **Algorithm**
- Generate a random variate Y from the distribution with PMF q_j .
- Generate a uniform random number $U \sim \text{Uniform}(0, 1)$.
- If $U < \frac{p_Y}{cq_Y}$, set $X = Y$ and stop. Otherwise, return to Step 1.

Specialized Algorithms for Common Distributions

Normal Distribution: The Polar Method generate two independent standard normal random variables. Algorithm

- Generate two independent uniform random numbers $U_1, U_2 \sim \text{Uniform}(0, 1)$.
- Set $V_1 = 2U_1 - 1$ and $V_2 = 2U_2 - 1$. These are uniform on $(-1, 1)$.
- Calculate $S = V_1^2 + V_2^2$.
- If $S \geq 1$, reject the pair (V_1, V_2) and return to step 1.

Otherwise, compute $X = V_1 \sqrt{\frac{-2 \ln(S)}{S}}$ and $Y = V_2 \sqrt{\frac{-2 \ln(S)}{S}}$

Poisson Distribution Exponential(λ) interarrival times $E_1, E_2, \dots$. Let $N = \max(n : \sum_{i=1}^n E_i \leq 1)$. Then $N \sim \text{Poisson}(\lambda)$. **Binomial Distribution**

a. Derive the CDF F(x).

```
def F(x):
    if x < 0:
        return 0.0
    elif x <= 1:
        return x**2
    else:
        return 1.0
```

b. Compute the inverse of F(x)

```
def Finv(u):
```

```
    return u**0.5
# c. Function to generate samples
def sample_triangular(n):
    U = np.random.uniform(0, 1, size=n) # n Uniform(0,1) samples
    X = Finv(U) # apply inverse transform
    return X
# d. Verification plot
import matplotlib.pyplot as plt
samples = sample_triangular(10000) # generate samples
plt.hist( # histogram
    samples,
    bins=30,
    density=True,
    edgecolor="black",
    facecolor="#C889C0",
    alpha=0.6,
    label="Samples")
# True PDF
x = np.linspace(0, 1, 200) # evenly spaced points in [0, 1]
f = 2*x
plt.plot(x, f, "k", lw=1, label="True PDF")
plt.legend()
plt.show()
```

```
# a. Implement the rejection sampling
def sample_triangular_reject(n):
    samples = []
    proposed = 0
    while len(samples) < n:
        y = np.random.uniform(0, 1) # proposal ~ Uniform(0, 1)
        u = np.random.uniform(0, 1) # uniform for acceptance
        # f(y) = 2y, g(y) = 1, c = 2=> accept if ???
        if u <= y:
            samples.append(y)
            proposed += 1
    return np.array(samples), proposed
```

```
# b. Generate 10,000 samples and plot a histogram with the true PDF.
import matplotlib.pyplot as plt
samples, proposed = sample_triangular_reject(10000)
plt.hist(
    samples,
    bins=30,
    density=True,
    edgecolor="black",
    facecolor="#C889C0",
    alpha=0.6,
    label="Samples (rejection)")
# True PDF f(x) = 2x on [0,1]
x = np.linspace(0, 1, 200)
f = 2* x
plt.plot(x, f, "k", lw=1.2, label="True PDF")
plt.title("Rejection sampling for f(x)=2x on [0,1]")
plt.xlabel("x")
plt.ylabel("density")
plt.legend()
plt.show()
```

```
# c. Estimate the acceptance rate.
accepted = len(samples)
print("Acceptance rate:", accepted / proposed) # should be close to 0.5
```

Monte Carlo

Suppose X is a random variable (or random vector) with known distribution. We are interested in

$$I = E[h(X)]$$

samples $X_1, \dots, X_N$ and computing

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n h(X_i)$$

- Each X_i is one independent simulation run.
- $h(X_i)$ is the observed output of that run.
- $\hat{I}_n$ is the Monte Carlo estimator of I .

Theorem. Law of Large Numbers (LLN) If $X_1, X_2, \dots, X_n$ are i.i.d. random variables with finite mean $E[X] = \mu$, then

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{a.s.} \mu \text{ as } n \rightarrow \infty$$

Algorithm (MC for LLN with Uniforms):

- Choose sample size n .
- Generate $U_i \sim \text{Uniform}(0, 1)$.
- Compute $\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n U_i$.
- Compare with exact mean.

Theorem. Central Limit Theorem (CLT) If $X_1, \dots, X_n$ are i.i.d. with mean μ and variance $\sigma^2 < \infty$, then

$$\frac{\hat{\mu}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \text{Normal}(0, 1).$$

Monte Carlo Estimation of Integrals

$$I = \int_0^1 g(x) \, dx$$

We can recognize this as $E[g(U)]$ where $U \sim \text{Uniform}(0, 1)$.

$$\hat{I} = \frac{1}{n} \sum_{i=1}^n g(U_i) \xrightarrow{a.s.} E[g(U)] = I \text{ as } n \rightarrow \infty$$

Algorithm (MC for 1D Integral):

- Choose n .
- Generate $U_i \sim U(0, 1)$.
- Compute $X_i = g(U_i)$.
- Average: $\hat{I}_n = \frac{1}{n} \sum_{i=1}^n X_i$.
- Report CI.

```
# a. Draw samples X, Y ~ Uniform(-1, 1) of size N
N = 10000
X = np.random.uniform(-1, 1, size=N)
Y = np.random.uniform(-1, 1, size=N)
# b. Count how many points are inside the unit circle (x^2 + y^2 <= 1)
inside_mask = (X**2 + Y**2) <= 1
inside_count = np.sum(inside_mask)
# c. Estimate pi
pi_hat = 4* inside_count / N
print(f"Estimated pi: {pi_hat:.6f} (from {inside_count} points inside)")
# d. Plot
plt.figure(figsize=(6,6))
plt.scatter(X[~inside_mask], Y[~inside_mask], s=5, alpha=0.5, label="Outside")
plt.scatter(X[inside_mask], Y[inside_mask], s=5, alpha=0.5, label="Inside")
plt.show()
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
from scipy import integrate
# a. Draw sample X ~ Uniform(0, 1) of size 10000
N = 10000
X = np.random.uniform(0, 1, size=N)
# b. Approximate the integral
f_vals = np.exp(-(X**2))
I_hat = f_vals.mean()
# c. SciPy quad (numerical integration)
ref_val, ref_err = integrate.quad(lambda t: np.exp(-(t**2)), 0.0, 1.0)
print(f"SciPy quad = {ref_val:.10f}")
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
N = 10000
Z = np.random.standard_normal(N)
p_hat = np.mean(Z > 2)
print(f"Monte Carlo p = {p_hat}")
```